

DEADLOCKS

- ❑ In multiprogramming system, several **processes** may **compete for a finite number of resources**.
- ❑ A **process requests resources** and if the resource is not available at that time, the process **enters a waiting state**. Sometimes, a waiting process is never again able to change state, because the resources it has requested are held other waiting processes. This situation is called Deadlock.

NORMAL MODE OF OPERATION

Under the normal mode of operation, a process may utilize a resource in only the following sequence.

□Request:

If the request cannot be granted immediately (for example, if the resource is being used by another process), then the requesting process must wait until it can acquire the resource.

□Use:

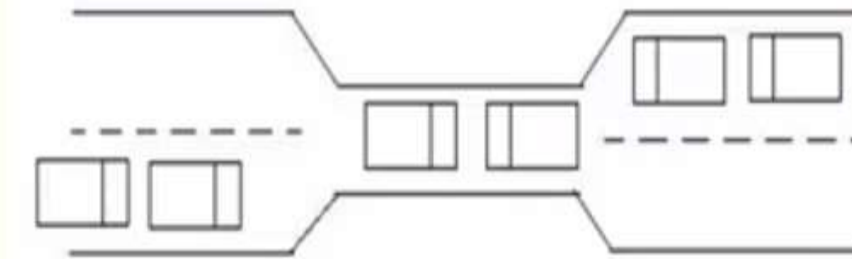
The process can operate on the resource (for example, if the resource is a printer, the process can print on the printer).

□Release:

The process releases the resource.

DEADLOCK EXAMPLES FROM DAILY LIFE

1. Bridge crossing example



- ☐ Traffic only in one direction.
- ☐ Each section of a bridge can be viewed as a resource.
- ☐ If a deadlock occurs, it can be resolved if one car backs up.
- ☐ Several cars may have to be backed up, if a deadlock occurs.

DEADLOCK EXAMPLES FROM DAILY LIFE (CONT.)

2. Two trains traveling towards each other on the same track.
3. A person going down a ladder, while another person is climbing up the ladder.
4. You can't get a job without experience; you can't get experience without a job.

NECESSARY CONDITIONS FOR DEADLOCK

A **deadlock can arise** if the following **four conditions** hold simultaneously in a system.

1. Mutual Exclusion.

2. Hold and wait.

3. No Preemption.

4. Circular wait.

MUTUAL EXCLUSION

At **least one resource** must be held in a **non-shareable mode**; that is only one process at a time can use the resource. If another process requests that resource, the requesting process must be delayed until the resource has been released.

HOLD AND WAIT

A process must be **holding at least one resource** and **waiting to acquire additional** resources that are currently being held by other processes.

NO PREEMPTION

Resources cannot be preempted;

a **resource can be released only voluntarily by the process holding it**, after that process has completed its task.

CIRCULAR WAIT

A set $\{P_0, P_1, P_2, \dots, P_n\}$ of waiting processes, such that **P_0 is waiting for a resource held by P_1** , P_1 is waiting for a resource held by $P_2, \dots, P_{n-1}$ is waiting for a resource held by P_n , and P_n is waiting for a resource held by P_0 .

RESOURCE-ALLOCATION GRAPH

- ❑ Deadlocks can be described in term of directed graph called a system resource allocation graph.
- ❑ Graph consists of a set of vertices V and set of edges E .
- ❑ The set of vertices are partitioned into two different types of nodes:
 - ❑ $P = \{P_1, P_2, \dots, P_n\}$, the set consists of all active processes in the system.
 - ❑ $R = \{R_1, R_2, \dots, R_n\}$, the set consists of all resource types in the system.

RESOURCE-ALLOCATION GRAPH(CONT.)

Request Edge:

- Directed edge from process P_i to resource type R_j
- A directed edge from process P_i to resource type R_j is denoted by $P_i \rightarrow R_j$. It signifies that process P_i has requested an instance of resource type R_j and currently waiting for that resource. A directed edge $P_i \rightarrow R_j$ is called *request edge*.

RESOURCE-ALLOCATION GRAPH(CONT.)

Assignment Edge:

- Directed edge from resource type R_j to process P_i
- A directed edge from resource type R_j to process P_i is denoted by $R_j \rightarrow P_i$. It signifies that an instance of resource type R_j has been allocated to process P_i . A directed edge $R_j \rightarrow P_i$ is called *assignment edge*.

RESOURCE-ALLOCATION GRAPH(CONT.)

□ Pictorially, we **represent** each **process P** as a **circle**,



□ and **resource R** as **rectangle**. Since resource may have more than one instance, we represent **each instance as a dot within the rectangle**.



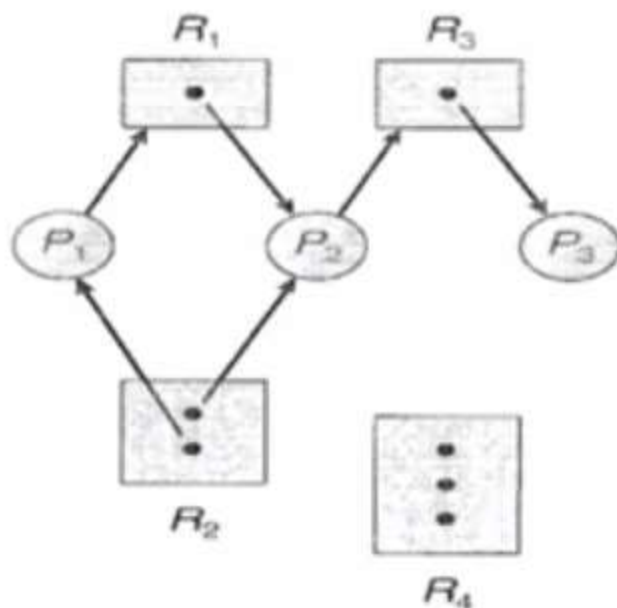
RESOURCE-ALLOCATION GRAPH(CONT.)

Example 1: Suppose that we have **sets of processes P**, set of resources R, and set of edges E:

$$\bullet P = \{P_1, P_2, P_3\}$$

$$\bullet R = \{R_1, R_2, R_3, R_4\}$$

$$\bullet E = \{P_1 \rightarrow R_1, P_2 \rightarrow R_3, R_1 \rightarrow P_2, R_2 \rightarrow P_2, R_2 \rightarrow P_1, R_3 \rightarrow P_3\}$$



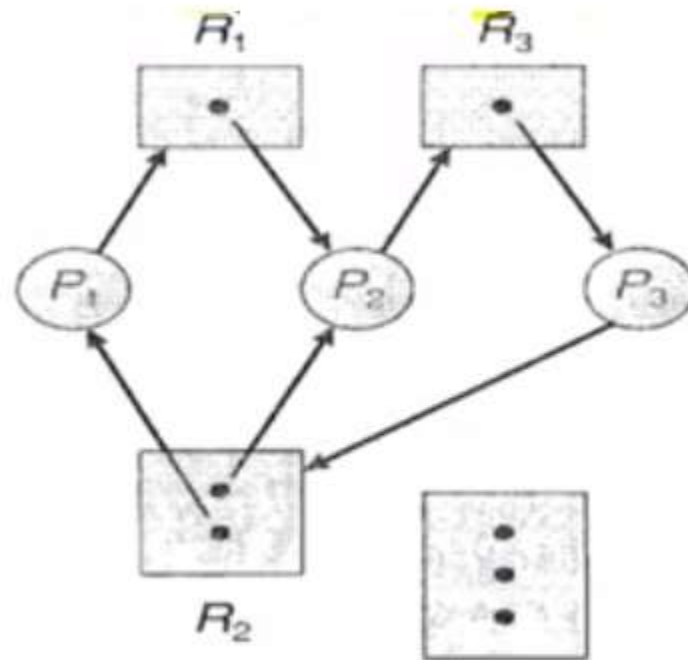
RESOURCE-ALLOCATION GRAPH(CONT.)

- ❑ Given the definition of a resource allocation graph, it can be shown that, **if the graph contains no cycles, then no process in the system is deadlocked.**
- ❑ If the graph does **contain a cycle**, then a **deadlock may exist**.
- ❑ If each **resource type has exactly one instance**, then a cycle implies that a deadlock that a **deadlock has occurred**.
- ❑ If each resource type **has several instances**, then a cycle does **not necessarily** imply that a **deadlock has occurred**.

RESOURCE-ALLOCATION GRAPH(CONT.)

Example 2: Suppose that we have **sets of processes P**, **set of resources R**, and set of edges E:

- $P = \{P_1, P_2, P_3\}$
- $R = \{R_1, R_2, R_3, R_4\}$
- $E = \{P_1 \rightarrow R_1, P_2 \rightarrow R_3, R_1 \rightarrow P_2, R_2 \rightarrow P_2, R_2 \rightarrow P_1, R_3 \rightarrow P_3, P_3 \rightarrow R_2\}$



RESOURCE-ALLOCATION GRAPH(CONT.)

Example 2: At this point, **two cycles** exist in the system:

- $P_1 \rightarrow R_1 \rightarrow P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$
- $P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_2$
- Processes **P_1 , P_2 , and P_3** are **deadlocked**.

Process P_2 is waiting for the resource R_3 , which is held by process P_3 . Process P_3 is waiting for either process P_1 or process P_2 to release resource R_2 . In addition, process P_1 is waiting for process P_2 to release resource R_1 .

RESOURCE-ALLOCATION GRAPH(CONT.)

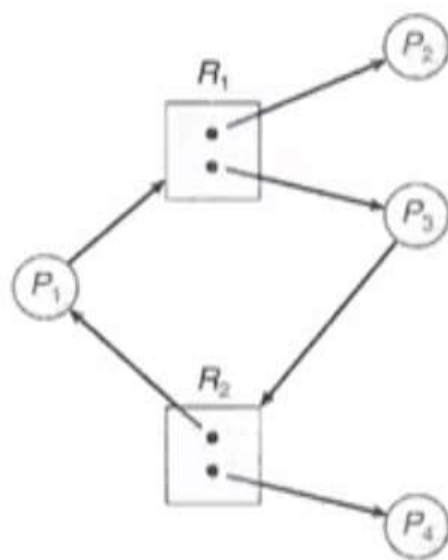
Example 3: Suppose that we have **sets of processes P**, **set of resources R**, and set of edges E:

- $P = \{P_1, P_2, P_3, P_4\}$
- $R = \{R_1, R_2\}$
- $E = \{P_1 \rightarrow R_1, R_1 \rightarrow P_2, R_1 \rightarrow P_3, P_3 \rightarrow R_2, R_2 \rightarrow P_1, R_2 \rightarrow P_4\}$

RESOURCE-ALLOCATION GRAPH(CONT.)

Example 3: In this example we also have a cycle

- $P_1 \rightarrow R_1 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$
- But **there is no deadlock**. Observe that process P_4 may release its instance of resource type R_2 . That resource can then be allocated to P_3 , and it will break the cycle.



DEADLOCK AVOIDANCE

Requires that the system has some additional *a priori* information available.

- ❑ Simplest and most useful model requires that each process declare the *maximum number* of resources of each type that it may need
- ❑ The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can *never* be a circular-wait condition
- ❑ A resource-allocation *state* is defined by the number of available and allocated resources, and the maximum demands of the processes

SAFE STATE

- ❑ When a process requests an available resource, the system must decide if immediate allocation leaves the system in a safe state

- ❑ A system is in a safe state only if there exists a safe sequence

- ❑ A sequence of processes $\langle P_1, P_2, \dots, P_n \rangle$ is a safe sequence for the current allocation state if, for each P_i , the resource requests that P_i can still make, can be satisfied by currently available resources plus resources held by all P_j with $j < i$.
- ❑ That is:
 - If the P_i resource needs are not immediately available, then P_i can wait until all P_j have finished
 - When P_j is finished, P_i can obtain needed resources, execute, return allocated resources, and terminate
 - When P_i terminates, P_{i+1} can obtain its needed resources, and so on

SAFE STATE (CONT.)

- ❑ If a system is in safe state $\Rightarrow$ no deadlocks
- ❑ If a system is in unsafe state $\Rightarrow$ possibility of deadlock
- ❑ Avoidance $\Rightarrow$ ensure that a system will never enter an unsafe state

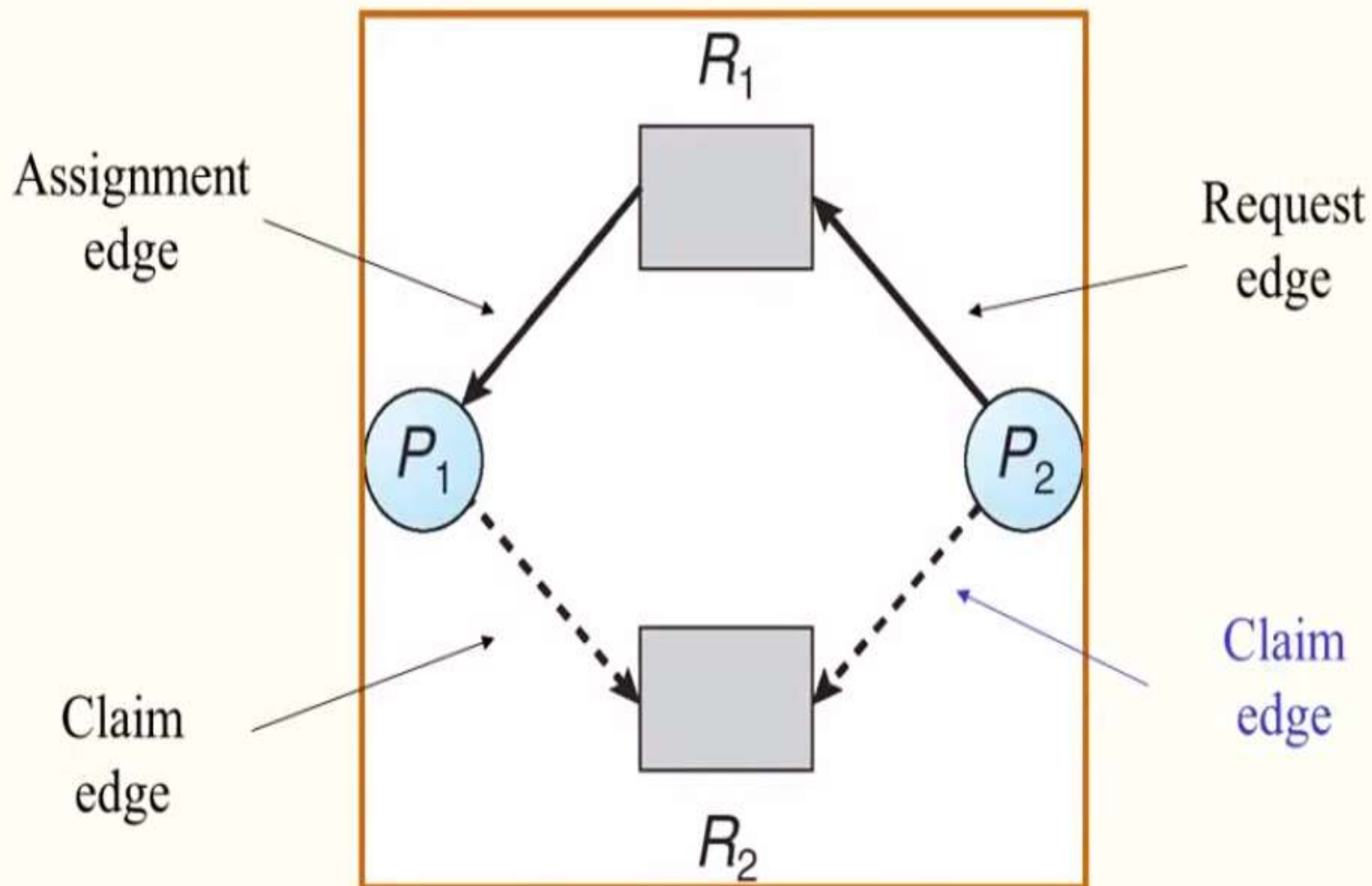
AVOIDANCE ALGORITHMS

- ❑ For a single instance of a resource type, use a resource-allocation graph scheme.
- ❑ For multiple instances of a resource type, use the banker's algorithm

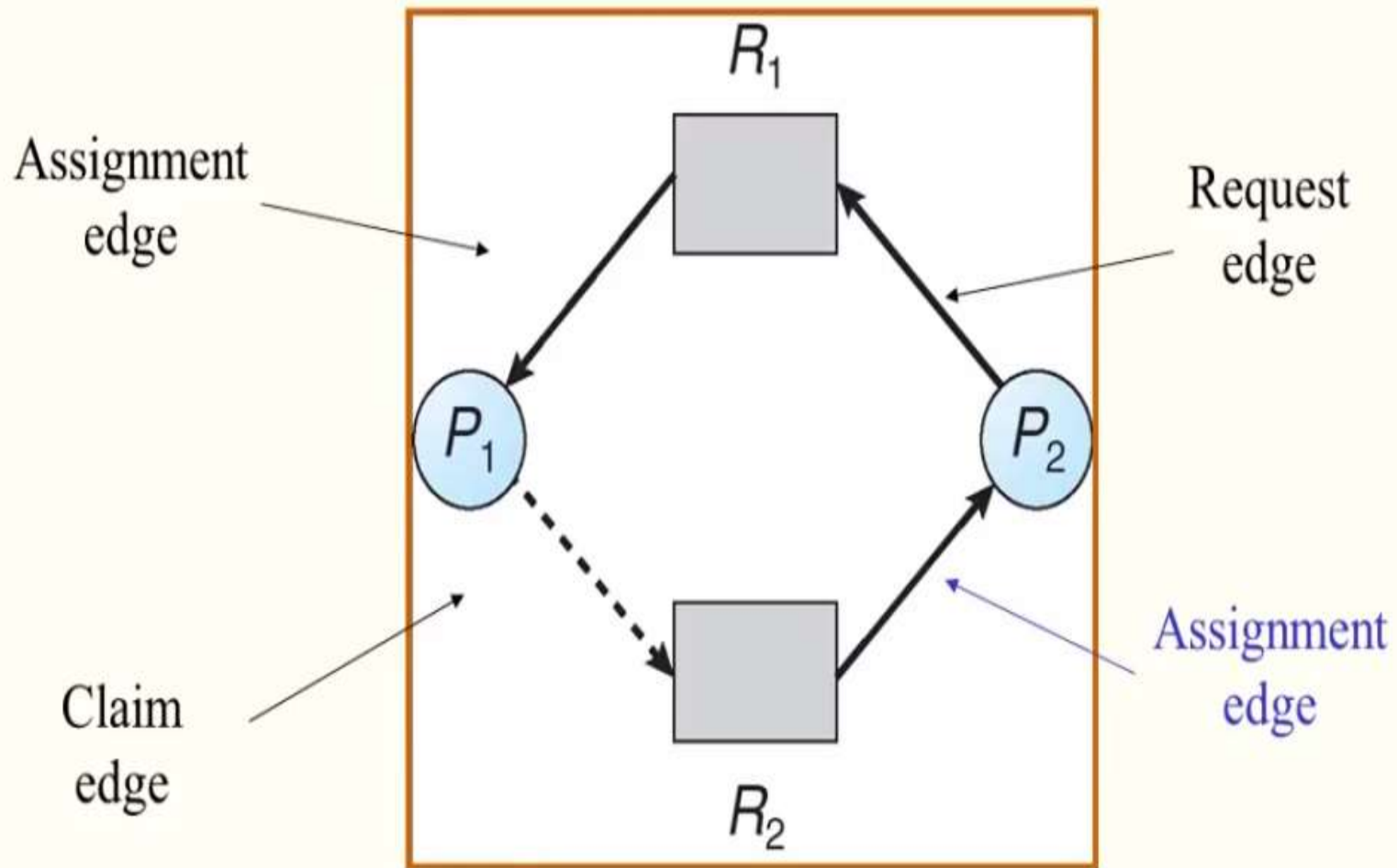
RESOURCE-ALLOCATION GRAPH SCHEME

- ❑ Introduce a new kind of edge called a claim edge
- ❑ *Claim edge* $P_i \text{ } \cdots \cdots \cdots \rightarrow R_j$ indicates that process P_j may request resource R_j ; which is represented by a dashed line
- ❑ A claim edge converts to a request edge when a process **requests** a resource
- ❑ A request edge converts to an assignment edge when the resource is **allocated** to the process
- ❑ When a resource is **released** by a process, an assignment edge reconverts to a claim edge
- ❑ Resources must be **claimed *a priori*** in the system

RESOURCE-ALLOCATION GRAPH WITH CLAIM EDGES



UNSAFE STATE IN RESOURCE-ALLOCATION GRAPH



RESOURCE-ALLOCATION GRAPH ALGORITHM

- ❑ Suppose that process P_i requests a resource R_j
- ❑ The request can be granted only if converting the request edge to an assignment edge does not result in the formation of a cycle in the resource allocation graph

EXAMPLE (SAFE AND UNSAFE STATE)

Consider a system with;

□ Total magnetic tape drives = 12

□ We have three processes P_0 , P_1 , and P_2 .

- Process P_0 requires 10 tape drives.
- Process P_1 requires 4 tape drives.
- Process P_2 requires 9 tape drives.

□ Suppose that, at time T_0 ;

- process P_0 is holding 5 tape drives.
- process P_1 is holding 2 tape drives.
- process P_2 is holding 2 tape drives.

□ Free tape drives = 3

EXAMPLE (SAFE AND UNSAFE STATE)_CONT.

Process	MaximumNeeds	Allocation
P ₀	10	5
P ₁	4	2
P ₂	9	2

□ At time T₀, the system is in a **safe state**.

- The sequence $\langle P_1, P_0, P_2 \rangle$ satisfies the safety condition.
 - Process **P₁ can immediately be allocated all its tape drives** and then return them (the system will then have five available tape drives).
 - Then process P₀ will get all its tape drives and return them (the system will then have ten available tape drives).
 - Finally process P₂ can get all its tape drives and return them (the system then have all twelve tape drives available).
- *Whenever a process requests a resource a resource can be allocated immediately or whether the process must wait. The request is granted only if the allocation leaves the system in a safe state.*

BANKER'S ALGORITHM

- ❑ This name is chosen because **this algorithm could be used in a banking system** to ensure that the bank never allocate its available cash in such a way that it could no longer satisfy the needs of all its customers.
- ❑ When a **new process enters** the system, it **must declare the maximum number of instances of each resource type that it may need**. This **number may not exceed the total number of resources in the system**. When a user requests a set of resources, the system must determine whether the **allocation of these resources, the system in a safe state**. If it will, the resources are allocated; otherwise, the process must wait until some other process releases enough resources.

BANKER'S ALGORITHM (EXAMPLE)

□ Consider a system with **five processes** P_0, P_1, P_2, P_3 , and P_4 . We have **three resource types** A, B, and C.

1. **Processes:** P_0, P_1, P_2, P_3, P_4

2. **Resources types:** A, B, C.

- Resource type **A** has 10 instances.
- Resource type **B** has 5 instances.
- Resource type **C** has 7 instances.

□ Suppose that, at **time T_0** , the following snapshot of the system has been taken:

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	3	3	2
P_1	2	0	0	3	2	2			
P_2	3	0	2	9	0	2			
P_3	2	1	1	2	2	2			
P_4	0	0	2	4	3	3			

BANKER'S ALGORITHM (EXAMPLE)

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P ₀	0	1	0	7	5	3	3	3	2
P ₁	2	0	0	3	2	2			
P ₂	3	0	2	9	0	2			
P ₃	2	1	1	2	2	2			
P ₄	0	0	2	4	3	3			

The contents of the **Need matrix** are as follows:

Process	Need		
	A	B	C
P ₀	7	4	3
P ₁	1	2	2
P ₂	6	0	0
P ₃	0	1	1
P ₄	4	3	1

We claim that the system is currently in a safe state. Indeed, the sequence $\langle P_1, P_3, P_4, P_2, P_0 \rangle$ satisfies the safety criteria.

BANKER'S ALGORITHM (EXAMPLE)

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P ₀	0	1	0	7	5	3	3	3	2
P ₁	2	0	0	3	2	2			
P ₂	3	0	2	9	0	2			
P ₃	2	1	1	2	2	2			
P ₄	0	0	2	4	3	3			

The contents of the **Need matrix**:

Process	Need		
	A	B	C
P ₀	7	4	3
P ₁	1	2	2
P ₂	6	0	0
P ₃	0	1	1
P ₄	4	3	1

- We will execute **P₁** first. P₁ makes **Request₁ (1, 2, 2)**
- To decide whether this request can be immediately granted, we first check that
Request₁ (1, 2, 2) ≤ Available

That is,

$$(1, 2, 2) \leq (3, 3, 2)$$

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P ₀	0	1	0	7	5	3	2	1	0
P ₁	3	2	2	3	2	2			
P ₂	3	0	2	9	0	2			
P ₃	2	1	1	2	2	2			
P ₄	0	0	2	4	3	3			

BANKER'S ALGORITHM (EXAMPLE)

P₁ request fulfilled (i.e. resources assigned to P₁)

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P ₀	0	1	0	7	5	3	1	1	0
P ₁	3	2	2	3	2	2			
P ₂	3	0	2	9	0	2			
P ₃	2	1	1	2	2	2			
P ₄	0	0	2	4	3	3			

The contents of the **Need matrix** will also updated;

Process	Need		
	A	B	C
P ₀	7	4	3
P ₁	0	0	0
P ₂	6	0	0
P ₃	0	1	1
P ₄	4	3	1

After P₁ will finish execution, it will release all allocated resources and it will be added to available.

Allocation Matrix and Available Matrix after P₁ finish execution:

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P ₀	0	1	0	7	5	3	4	3	2
P ₁	0	0	0	3	2	2			
P ₂	3	0	2	9	0	2			
P ₃	2	1	1	2	2	2			
P ₄	0	0	2	4	3	3			

BANKER'S ALGORITHM (EXAMPLE)

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P ₀	0	1	0	7	5	3	4	3	2
P ₁	0	0	0	3	2	2			
P ₂	3	0	2	9	0	2			
P ₃	2	1	1	2	2	2			
P ₄	0	0	2	4	3	3			

The contents of the **Need matrix**;

Process	Need		
	A	B	C
P ₀	7	4	3
P ₁	0	0	0
P ₂	6	0	0
P ₃	0	1	1
P ₄	4	3	1

- Now we will execute P₃. P₃ makes **Request₁ (0, 1, 1)**
 - To decide whether this request can be immediately granted, we first check that **Request₁ (0, 1, 1) ≤ Available**
- That is,
- $$(0, 1, 1) \leq (4, 3, 2)$$

P₃ request fulfilled (i.e. resources assigned to P₃)

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P ₀	0	1	0	7	5	3	4	2	1
P ₁	0	0	0	3	2	2			
P ₂	3	0	2	9	0	2			
P ₃	2	2	2	2	2	2			
P ₄	0	0	2	4	3	3			

BANKER'S ALGORITHM (EXAMPLE)

P_3 request fulfilled (i.e. resources assigned to P_3)

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	4	2	1
P_1	0	0	0	3	2	2			
P_2	3	0	2	9	0	2			
P_3	2	2	2	2	2	2			
P_4	0	0	2	4	3	3			

The contents of the **Need matrix** will also be updated ;

Process	Need		
	A	B	C
P_0	7	4	3
P_1	0	0	0
P_2	6	0	0
P_3	0	0	0
P_4	4	3	1

After P_3 will finish execution, it will release all allocated resources and it will be added to available.

Allocation Matrix and Available Matrix after P_3 finish execution:

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	6	4	3
P_1	0	0	0	3	2	2			
P_2	3	0	2	9	0	2			
P_3	0	0	0	2	2	2			
P_4	0	0	2	4	3	3			

BANKER'S ALGORITHM (EXAMPLE)

Apply same method for other processes